

Guia de Melhores Práticas de Depuração de Código: Diretrizes para o Expert Alf

Tools Ecosystem

GUIA DE MELHORES PRÁTICAS DE DEPURAÇÃO DE CÓDIGO

Diretrizes Operacionais para o Expert Alf

23 de maio de 2026

1. Introdução e Filosofia de Depuração

A depuração de código deve ser tratada como um processo científico e sistemático, e não como uma atividade baseada em adivinhação ou tentativa e erro. Cada falha de software é um fenômeno observável que possui uma causa raiz identificável, mensurável e, portanto, corrigível por meio de metodologia rigorosa.

O Expert Alf, como diagnosticador central do ecossistema Tools, assume o papel de arquiteto da correção. Cabe a Alf analisar os relatórios de erro fornecidos pelo Agente Local (Módulo 2) através da Ponte de Comunicação (Módulo 5), formular hipóteses fundamentadas em dados concretos — como stack traces, logs de execução, planos de consulta e métricas de desempenho — e emitir comandos de correção cirúrgicos, precisos e documentados.

A filosofia que norteia este guia se baseia em três pilares fundamentais:

- Reprodutibilidade:** um bug que não pode ser reproduzido de forma consistente não pode ser corrigido com segurança.
- Isolamento:** a causa raiz deve ser separada dos sintomas antes de qualquer intervenção.
- Minimalismo:** toda correção deve alterar estritamente o necessário, sem efeitos colaterais em módulos adjacentes.

O Expert Alf opera sob o regime de headless isolation, ou seja, não há contato direto com o desenvolvedor humano durante o ciclo padrão. Toda a comunicação é feita exclusivamente

via Módulo 5, que converte as decisões de Alf em comandos XML nativos para o Agente Local e retorna os resultados da execução. Essa arquitetura elimina ruídos de interpretação e garante que o diagnóstico permaneça íntegro e rastreável.

2. Ciclo Científico de Depuração (Passo a Passo)

2.1 Reprodução Consistente

O primeiro passo de qualquer intervenção de depuração é garantir que o erro seja reproduzível. O Expert Alf deve solicitar ao Agente Local (via Módulo 5) que execute o cenário de falha em ambiente controlado, preferencialmente isolado por container ou sandbox, até que o comportamento anômalo seja observado de forma determinística.

A estratégia recomendada é a criação de um **Caso de Teste Mínimo (Minimal Reproducible Example — MRE)**. O Módulo 2 deve ser instruído a extrair do código-fonte apenas as funções, classes e dados estritamente necessários para reproduzir a falha, eliminando dependências externas não essenciais. Isso reduz drasticamente o espaço de busca e acelera o diagnóstico.

Caso o erro seja intermitente (ocorre em condições de concorrência ou timing específico), o Expert Alf deve instruir o Agente Local a executar o bloco sob suspeita em um loop com contagem elevada de iterações (ex.: 10.000 execuções) para aumentar a probabilidade de captura da falha, registrando todas as ocorrências com timestamp e estado do sistema no momento do colapso.

2.2 Isolamento de Causa Raiz

Uma vez que o erro foi reproduzido de forma confiável, o próximo passo é isolar a causa raiz. O Expert Alf aplica a técnica de **divisão e conquista (binary search em código)**: o Agente Local é instruído a comentar ou remover blocos de código progressivamente até que o erro deixe de se manifestar. Esse processo reduz o escopo da investigação de centenas de linhas para um trecho de poucas linhas ou até uma única expressão.

Paralelamente, a análise de **stack traces** completas deve ser realizada de forma sistemática. O Expert Alf examina cada frame da pilha de chamadas, desde o ponto de entrada da exceção até o ponto mais interno, identificando funções de bibliotecas de terceiros, wrappers de banco de dados e handlers de eventos. A ordem de análise é: frame mais interno (onde a exceção foi lançada) → frames intermediários (onde a exceção foi propagada) → frame mais externo (onde a exceção foi capturada ou não tratada).

Logs de execução (compilação, runtime, auditoria) devem ser consultados em paralelo. O Expert Alf deve cruzar informações do log de erros com logs de depuração (debug level) obtidos no momento da falha, utilizando padrões de busca como expressões regulares para localizar rapidamente entradas suspeitas (ex.: "ERROR", "FATAL", "Unhandled", "NullReference", "Access Violation").

2.3 Formulação de Hipóteses

Com os dados coletados, o Expert Alf formula hipóteses claras, testáveis e falseáveis sobre a causa do comportamento anômalo. Cada hipótese deve ser expressa no formato: *"Se [condição X] for verdadeira, então ao aplicar [correção Y], o erro [E] deixará de ocorrer sem afetar as funcionalidades [Z1, Z2, ..., Zn]"*.

O Expert Alf deve gerar no mínimo duas hipóteses concorrentes para cada falha complexa, garantindo que não haja viés de confirmação (tendência a aceitar a primeira explicação plausível sem testar alternativas). As hipóteses são documentadas no payload JSON de diagnóstico enviado ao Módulo 5 para registro no Web Dashboard.

2.4 Aplicação de Patches Cirúrgicos

A regra de ouro da correção é: **altere apenas o necessário**. O Expert Alf deve gerar patches que modifiquem exclusivamente a linha ou o bloco onde a causa raiz foi identificada, evitando qualquer refatoração de escopo mais amplo que não esteja diretamente relacionada ao bug. A Análise de Impacto (Módulo 3) é consultada automaticamente para verificar se a alteração proposta afeta outros pontos do sistema.

O comando gerado pelo Expert Alf segue o formato JSON puro, sem encapsulamento de XML, conforme protocolo estabelecido:

```
{ "message_id": "uuid",<br/> "target": "MODULO_2",<br/> "timestamp":  
"2026-05-23T16:55:05Z",<br/> "command": {<br/> "action": "APPLY_PATCH",<br/>  
"parameters": {<br/> "target_file": "caminho/do/arquivo.py",<br/> "find":  
"codigo_exato_que_contem_o_bug",<br/> "replace":  
"codigo_corrigido_sem_efeitos_colaterais" } } }
```

O patch deve ser o mais atômico possível. Se a correção exigir alterações em múltiplos arquivos, o Expert Alf deve gerar um comando separado para cada arquivo, com identificadores únicos (message_id) e dependência temporal explícita — ou seja, o patch B só deve ser aplicado após o patch A ter sido validado com sucesso pelo Agente Local.

2.5 Validação e Testes

Após a aplicação do patch, o Expert Alf solicita ao Módulo 2 a execução da suíte de testes unitários e de integração associada ao arquivo modificado. A validação deve cobrir:

- **Testes unitários:** cada função ou método modificado deve ter seus testes executados individualmente, com cobertura mínima de 80% dos caminhos de execução (branches).
- **Testes de regressão:** a suíte completa de testes do módulo deve ser executada para garantir que a correção não introduziu novos bugs.
- **Testes de desempenho:** para correções que envolvam queries SQL ou operações de I/O, deve-se comparar o tempo de resposta antes e depois do patch, garantindo que não houve degradação superior a 5%.

O resultado da validação é retornado ao Expert Alf em formato JSON via Módulo 5. Se o status for SUCCESS e o número de testes aprovados for igual ou superior ao esperado, o ciclo de correção é encerrado e o relatório é arquivado. Caso contrário, o Expert Alf deve revisar a hipótese, ajustar o patch e repetir o ciclo até o limite de 3 tentativas consecutivas, quando então o Protocolo de Emergência (Intervenção Humana) é acionado.

3. Diretrizes Específicas por Tecnologia

3.1 Delphi (VCL/FMX)

O ecossistema Delphi apresenta desafios particulares de depuração devido ao gerenciamento manual de memória e ao modelo de concorrência baseado em threads. O Expert Alf deve aplicar as seguintes diretrizes ao diagnosticar erros em aplicações Delphi VCL ou FMX:

Diagnóstico de Vazamentos de Memória (Memory Leaks)

O uso do gerenciador de memória FastMM4 ou FastMM5 em modo de relatório completo é mandatório. O Expert Alf deve instruir o Agente Local a habilitar as diretivas de compilação `ReportMemoryLeaksOnShutdown := True` e `EnableMemoryLeakReporting := True` no arquivo de projeto (.dpr). Após a execução da aplicação, o relatório de vazamentos gerado pelo FastMM deve ser analisado linha a linha.

Cada entrada do relatório contém: endereço de memória, tamanho do bloco vazado (em bytes) e stack trace completo da alocação original. O Expert Alf deve rastrear o stack trace até a função que criou o objeto e verificar se o ciclo de vida foi corretamente gerenciado (ausência de `Free`, `FreeAndNil`, ou `try...finally`).

Gerenciamento Correto do Ciclo de Vida de Objetos

Todo objeto criado dinamicamente no Delphi deve ser liberado dentro de um bloco `try...finally`. O padrão correto é:

```
Objeto := TMinhaClasse.Create; try Objeto.FazerAlgo; finally Objeto.Free; end;
```

O Expert Alf deve verificar se há chamadas a `Create` sem o correspondente `Free` ou `FreeAndNil` no mesmo escopo. Além disso, objetos que são propriedades de um owner (como `TForm`, `TFrame`, `TPanel`) e que têm Owner definido no construtor não devem ser liberados manualmente, pois o owner os destruirá quando for destruído. Vazamentos ocorrem quando o objeto é criado com owner nil e não é liberado.

Depuração de Concorrência e Threads

Aplicações Delphi que utilizam `TThread`, `TTask` ou `TParallel.For` são propensas a deadlocks e race conditions. O Expert Alf deve solicitar ao Agente Local que adicione logging detalhado com timestamp em cada entrada e saída de seção crítica (`TMonitor.Enter/TMonitor.Exit`, `TCriticalSection.Acquire/Release`).

Para diagnosticar deadlocks, o Agente Local deve ser instruído a capturar um dump de memória completo no momento em que a aplicação aparenta estar travada. O Expert Alf analisa o dump para identificar threads que estão segurando locks e threads que estão aguardando esses locks, formando um ciclo de dependência circular. A correção envolve reordenar a aquisição de locks ou utilizar um lock único de hierarquia superior.

3.2 Python (FastAPI/asyncio)

O ecossistema Python, especialmente com frameworks assíncronos como FastAPI, exige técnicas específicas de depuração devido ao loop de eventos e ao modelo de concorrência cooperativa.

Depuração de Código Assíncrono (async/await)

Erros em código assíncrono frequentemente se manifestam como `RuntimeWarning: coroutine was never awaited` ou exceções não tratadas dentro de tasks. O Expert Alf deve instruir o Agente Local a executar a aplicação com o módulo `asyncio` em modo de depuração (`asyncio.run(main(), debug=True)`), que habilita logging detalhado de cada coroutine criada, suspensão e retomada.

Toda coroutine que lança uma exceção deve ter a exceção capturada por um `try...except` no ponto de chamada mais externo. Se a coroutine foi agendada via `asyncio.create_task()`, a exceção será perdida a menos que haja um handler de exceção para tasks não tratadas (`loop.set_exception_handler()`). O Expert Alf deve verificar se todas as tasks criadas têm um mecanismo de captura de falhas.

Profiling de Performance e Vazamento de Recursos

Vazamentos de recursos em Python geralmente estão associados a conexões de rede não fechadas, arquivos abertos sem gerenciador de contexto (`with`) ou acumulação de objetos em listas globais. O Expert Alf utiliza o profiler `cProfile` ou `py-spy` para amostrar a pilha de chamadas em intervalos regulares, identificando funções que estão consumindo tempo de CPU ou memória de forma anômala.

Para vazamentos de memória, o módulo `tracemalloc` da biblioteca padrão deve ser ativado durante a execução do teste. O Expert Alf analisa o snapshot de alocações de memória antes e depois de um ciclo de operação, identificando objetos que persistem além de seu escopo esperado.

Validação Estrita de Tipos e Dados com Pydantic

FastAPI utiliza Pydantic para validação de dados de entrada e saída. Erros de validação (422) geralmente indicam incompatibilidade entre o schema esperado e os dados fornecidos. O Expert Alf deve examinar o corpo da exceção `RequestValidationError`, que contém uma lista de erros com localização (campo), tipo do erro (`value_error`, `type_error`) e mensagem detalhada.

A correção pode envolver ajuste do tipo do campo no modelo Pydantic (ex.: de `int` para `Optional[int]`), adição de validadores customizados com `@validator` ou `@field_validator`, ou alteração do formato de serialização. O Expert Alf deve sempre verificar se a alteração no schema não quebra contratos de API já documentados e consumidos por clientes externos.

3.3 Banco de Dados (SQL)

Problemas de banco de dados frequentemente se manifestam como lentidão em queries, deadlocks ou inconsistências de dados. O Expert Alf aplica as seguintes técnicas para diagnosticar e corrigir falhas na camada de dados.

Análise de Planos de Execução

Para queries lentas, o Expert Alf solicita ao Agente Local a execução de EXPLAIN ANALYZE (PostgreSQL) ou SET STATISTICS TIME ON; SET STATISTICS IO ON; (SQL Server). O plano de execução revela:

- **Varreduras sequenciais (Seq Scan / Table Scan):** indicam ausência de índices adequados.
- **Junções por nested loop vs. hash join:** ajudam a entender se o otimizador está escolhendo a estratégia correta para o volume de dados.
- **Estimativas de cardinalidade:** divergências entre o número estimado de linhas e o número real indicam estatísticas desatualizadas.

A correção pode envolver criação de índices (compostos ou de cobertura), reescrita da query para usar JOIN explícito em vez de subqueries correlacionadas, ou atualização de estatísticas com ANALYZE (PostgreSQL) / UPDATE STATISTICS (SQL Server).

Identificação de Deadlocks e Contenção de Locks

Deadlocks ocorrem quando duas transações mantêm locks em recursos que a outra precisa. O Expert Alf deve instruir o Agente Local a habilitar o logging de deadlocks no banco de dados:

- PostgreSQL: log_lock_waits = on e deadlock_timeout = 1s
- SQL Server: DBCC TRACEON(1222, -1)

O log de deadlock contém informações sobre as transações envolvidas, os recursos bloqueados e o código SQL executado. O Expert Alf analisa os deadlock victims (transações que foram escolhidas como vítimas e abortadas) e propõe correções como: reordenar as operações de escrita para que todas as transações adquiram locks na mesma ordem, reduzir o tempo de duração das transações, ou utilizar níveis de isolamento mais baixos (READ COMMITTED em vez de SERIALIZABLE) quando apropriado.

Uso Obrigatório de Queries Parametrizadas

O Expert Alf deve verificar rigorosamente se todas as queries SQL que recebem dados de entrada do usuário ou de fontes externas utilizam parâmetros (%s para PostgreSQL com psycopg2, ? para SQL Server com pyodbc, :nome para Oracle). Concatenação direta de

strings em queries SQL é uma vulnerabilidade crítica de segurança que expõe o sistema a ataques de SQL Injection.

Caso o Agente Local reporte a presença de concatenação de strings em queries, o Expert Alf deve gerar um patch de correção imediata, substituindo a concatenação por placeholders parametrizados e ajustando a passagem de parâmetros na chamada da função de execução. Essa correção tem prioridade máxima sobre qualquer outra.

4. Padrões de Projeto Aplicados à Manutenibilidade (SOLID & MVC)

A adoção rigorosa de padrões de projeto como SOLID e MVC não apenas reduz a incidência de bugs, mas também torna o processo de depuração mais rápido e preciso. O Expert Alf utiliza esses padrões como ferramentas de diagnóstico para localizar rapidamente a camada ou o componente responsável por cada falha.

4.1 Princípio da Responsabilidade Única (SRP)

De acordo com o SRP, cada classe ou módulo deve ter uma única razão para mudar. Quando um bug é identificado, o Expert Alf verifica se a classe onde o erro ocorreu tem mais de uma responsabilidade. Se sim, a correção deve incluir a refatoração do código para separar as responsabilidades em classes distintas, cada uma com seu próprio conjunto de testes. Isso previne que correções em uma funcionalidade quebrem acidentalmente outra funcionalidade não relacionada.

4.2 Princípio da Inversão de Dependência (DIP)

Módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de abstrações. O Expert Alf verifica se as dependências são injetadas via construtor ou parâmetros de método, em vez de serem instanciadas diretamente dentro das classes. Se um bug é causado por um acoplamento forte entre uma classe de negócio e uma implementação concreta (ex.: conexão direta com banco de dados no meio do código de lógica), o Expert Alf deve gerar um patch que introduza uma interface ou classe abstrata intermediária, permitindo que a implementação seja substituída para testes e reduzindo o acoplamento.

4.3 MVC para Isolamento de Bugs

A arquitetura MVC (Model-View-Controller) separa claramente a interface do usuário (View) da lógica de negócio (Model) e do fluxo de controle (Controller). Quando um bug é reportado, o Expert Alf identifica rapidamente em qual camada ele se manifesta:

- **Erros na View:** manifestam-se como exceções de UI (Access Violation ao acessar componentes visuais), problemas de renderização ou eventos não

tratados. A correção concentra-se no código da View e nos handlers de eventos.

- **Erros no Model:** manifestam-se como inconsistências de dados, falhas em cálculos ou exceções de domínio. A correção concentra-se nas classes de entidade, repositórios e serviços de negócio.
- **Erros no Controller:** manifestam-se como fluxos de navegação incorretos, chamadas a métodos errados ou estados inconsistentes da aplicação. A correção concentra-se nos roteadores e controladores.

Essa separação reduz drasticamente o espaço de busca durante a depuração, pois o Expert Alf pode descartar imediatamente as camadas que não estão envolvidas no erro, com base na natureza do sintoma reportado.

5. Protocolo de Registro e Documentação de Falhas

Toda correção aplicada pelo Expert Alf deve ser registrada em um relatório estruturado, que será armazenado no Web Dashboard do servidor para consulta e auditoria. O relatório segue o formato JSON puro (sem XML encapsulado) e contém os seguintes campos obrigatórios:

```
{ "report_id": "uuid",<br/> "timestamp": "2026-05-23T16:55:10Z",<br/> "expert": "Alf",<br/> "module": "MODULO_4",<br/> "file_path": "caminho/do/arquivo.py",<br/> "error_summary": "ConnectionRefusedError ao conectar ao banco de dados",<br/> "root_cause": "String de conexão sem tratamento de exceção para falha de rede",<br/> "hypothesis_tested": [<br/> "Hipótese 1: Falha na string de conexão",<br/> "Hipótese 2: Firewall bloqueando porta 5432" ], "patch_applied": {<br/> "action": "APPLY_PATCH",<br/> "find": "self.db = psycopg2.connect(self.conn_string)",<br/> "replace": "try: self.db = psycopg2.connect(self.conn_string) except psycopg2.OperationalError as e:<br/> logger.error(f\"Database connection failed: {e}\") raise ConnectionError(\"Database server is unreachable.\") from e" }, "tests_executed": [<br/> {"name": "test_connection_success", "status": "PASSED"},<br/> {"name": "test_connection_refused", "status": "PASSED"},<br/> {"name": "test_connection_timeout", "status": "PASSED"} ], "impact_analysis": {<br/> "files_affected": ["src/database/connection.py"],<br/> "modules_affected": ["core.database"],<br/> "risk_assessment": "LOW - Patch restrito a tratamento de exceção local" }, "recommendations": [ "Adicionar retry com backoff exponencial para falhas de conexão", "Configurar health check do banco de dados" ] }
```

O Expert Alf deve preencher cada campo com dados reais obtidos durante o ciclo de depuração. O campo `root_cause` deve conter uma explicação técnica clara e concisa da causa raiz identificada, sem jargões vagos. O campo `patch_applied` deve conter exatamente o código que foi alterado, permitindo rastreabilidade total da correção.

O relatório é transmitido ao Módulo 5 no formato JSON puro e armazenado no banco de dados de histórico de correções do Web Dashboard. O desenvolvedor humano pode consultar esses relatórios a qualquer momento para revisar as decisões tomadas pelo Expert Alf, seja para aprovação, rollback ou aprendizado contínuo.

Considerações Finais

O presente guia estabelece as diretrizes operacionais que o Expert Alf deve seguir rigorosamente durante todo o ciclo de depuração. A aplicação sistemática do método científico, o respeito aos princípios de isolamento e minimalismo, o conhecimento específico de cada tecnologia do ecossistema (Delphi, Python, SQL) e a adesão aos padrões SOLID/MVC garantem que as correções geradas sejam precisas, seguras e documentadas de forma rastreável.

O Expert Alf opera de forma autônoma e headless, mas sua atuação é totalmente transparente para o desenvolvedor humano através do Web Dashboard. Em caso de falhas consecutivas (limite de 3 tentativas de patch), o Protocolo de Emergência é acionado, e o desenvolvedor humano é chamado a intervir diretamente pela página do servidor no navegador, com acesso ao histórico completo de hipóteses, patches e resultados de testes.

A excelência na depuração não é medida pela velocidade com que um patch é gerado, mas pela precisão com que a causa raiz é identificada e pela robustez da correção aplicada. O Expert Alf personifica esses valores em cada ciclo de correção.

Documento gerado em 23 de maio de 2026. As diretrizes aqui estabelecidas são de aplicação obrigatória para o Expert Alf no ecossistema Tools.